

Lectura 8

Resolución de Sistemas de Ecuaciones Lineales

Métodos iterativos

Dr. Luciano Ponzellini Marinelli
lucianoponzellini@uca.edu.ar

Facultad de Química e Ingeniería
Rosario, Argentina

Notas de clase
30 Septiembre 2025





Organización

INTRODUCCIÓN

MÉTODOS DE JACOBI

MÉTODO DE GAUSS-SEIDEL

MÉTODO DE SOR

CONVERGENCIA

CONDICIONAMIENTO

INTRODUCCIÓN

Normas en espacios vectoriales

Definición: Sea $(\mathbb{V}, +, \cdot)$ un espacio vectorial con sus operaciones $+$ suma de vectores y $\cdot$ producto por escalares (en $\mathbb{R}$ o $\mathbb{C}$).

Una **norma** $\|\cdot\|$ es una función a valores reales que satisface:

$$\begin{aligned}
 \|\mathbf{x}\| &> 0 && \forall \mathbf{x} \neq \mathbf{0} \\
 \|\mathbf{x}\| &= 0 && \text{sí y solo sí } \mathbf{x} = \mathbf{0} \\
 \|\lambda \mathbf{x}\| &= |\lambda| \|\mathbf{x}\| && \lambda \in \mathbb{R} \\
 \|\mathbf{x} + \mathbf{y}\| &\leq \|\mathbf{x}\| + \|\mathbf{y}\| && (\text{desigualdad triangular})
 \end{aligned}$$

Decimos que $(\mathbb{V}, \|\cdot\|)$ es un **espacio vectorial normado**.

- La noción de norma generaliza el concepto de *longitud*
- Trabajaremos con $\mathbb{V} = \mathbb{R}^n$ (vectores) o $\mathbb{V} = \mathbb{R}^{m \times n}$ (matrices)

Normas vectoriales en $\mathbb{R}^n$

- Sea $\mathbf{x} = (x_1, \dots, x_n) \in \mathbb{R}^n$. Las **p -normas** son:

$$\|\mathbf{x}\|_p = \left(\sum_{i=1}^n |x_i|^p \right)^{1/p} \quad 1 \leq p \leq \infty$$

- En particular nos interesan:

$$p = 1 \quad \mapsto \quad \|\mathbf{x}\|_1 = \sum_{i=1}^n |x_i| \quad \text{(Norma 1)}$$

$$p = 2 \quad \mapsto \quad \|\mathbf{x}\|_2 = \left(\sum_{i=1}^n |x_i|^2 \right)^{1/2} \quad \text{(Norma Euclídea)}$$

$$p = \infty \quad \mapsto \quad \|\mathbf{x}\|_\infty = \max_{i=1, \dots, n} |x_i| \quad \text{(Norma infinito)}$$

Comparación normas vectoriales

- Sea la bola unitaria en la p -norma:

$$B = \{\mathbf{x} \in \mathbb{V} : \|\mathbf{x}\|_p \leq 1\}$$

- Geométricamente:

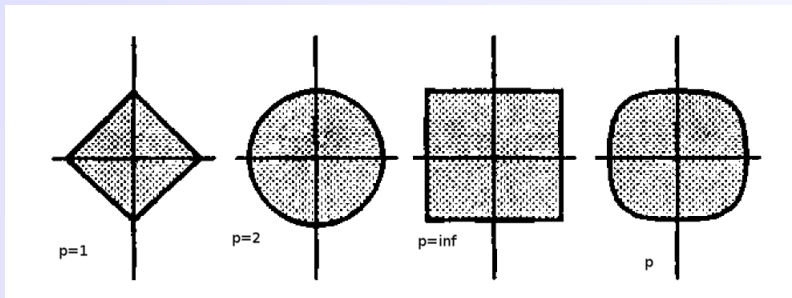


Figura: Bola unitaria en distintas normas

Normas vectoriales en MATLAB/Octave/Scilab

- En MATLAB/Octave/Scilab tenemos predefinida la función **norm(x,p)** que calcula la $\|x\|_p$ de un vector

```
>> x=(1:4)/5
x = 0.2000 0.4000 0.6000 0.8000
```

```
>> norm1=norm(x,1)
norm1 = 2
```

```
>> norm2=norm(x)           (por defecto en la norma  $\|x\|_2$ )
norm2 = 1.0954
```

```
>> norminf=norm(x,inf)     (norm(x,'inf') en Scilab)
norminf = 0.8000
```

Implementación de norma vectorial: *normap.m/.sci*

```
function np = normap(x,p)
% Cálculo de norma p-ésima de un vector
% Datos:  - x vector nx1
%          - p norma a calcular (1<=p<= inf)
% Resultado:
%          - np norma p-ésima de x
if p == inf
    np = max(abs(x));
else
    np = (sum(abs(x).^p)) ^(1/p);
end
end
```


Normas matriciales en $\mathbb{R}^{m \times n}$

- Dada $\mathbf{A} = (a_{ij})_{i,j=1,\dots,n}$, algunas normas matriciales son:

$$p = 1 \quad \mapsto \quad \|\mathbf{A}\|_1 = \max_{j=1,\dots,n} \sum_{i=1}^n |a_{ij}| \quad (\text{Norma 1})$$

$$p = 2 \quad \mapsto \quad \|\mathbf{A}\|_2 = \left(\sum_{i,j=1}^n |a_{ij}|^2 \right)^{1/2} \quad (\text{Norma Euclídea})$$

$$p = \infty \quad \mapsto \quad \|\mathbf{A}\|_\infty = \max_{i=1,\dots,n} \sum_{j=1}^n |a_{ij}| \quad (\text{Norma infinito})$$

- La norma Euclídea no siempre satisface la **condición de compatibilidad**

$$\|\mathbf{Ax}\|_p \leq \|\mathbf{A}\|_p \|\mathbf{x}\|_p$$

Metódos iterativos

- Una técnica fundamental de resolución en los Métodos Numéricos es la **iteración**.
- Se repite un proceso determinado para obtener una sucesión de valores hasta un resultado suficientemente aproximado que idealmente converge a la solución.
- Se necesita:
 - un **esquema iterativo** (regla, fórmula, función),
 - una **condición inicial**,
 - un **tolerancia de corte**.
- Los métodos iterativos presentan ventajas sobre los métodos directos en velocidad y memoria

Estudiaremos el proceso de iteración que consiste en sustituir repetidamente un valor obtenido en una misma fórmula

Metódos iterativos

- Sea G la función para calcular los términos sucesivos:

$$\mathbf{x} = G(\mathbf{x}).$$

siendo $\mathbf{p}_0$ la *aproximación inicial*.

- Generamos una sucesión $\{\mathbf{p}_k\}$ mediante el proceso iterativo

$$\mathbf{p}_{k+1} = G(\mathbf{p})_k, \quad k \geq 0.$$

- Deseamos que cuando $k \rightarrow \infty$

$$\mathbf{p}_k \rightarrow \mathbf{r}$$

siendo $\mathbf{r}$ la solución de nuestro problema numérico.

¿Cuándo cortamos? Cuando la tolerancia sea la deseada

MÉTODO DE JACOBI

Metódos iterativos: *Jacobi*

- Consideremos el SEL:

$$\begin{cases} 4x_1 - x_2 + x_3 = 7 \\ 4x_1 - 8x_2 + x_3 = -21 \\ -2x_1 + x_2 + 5x_3 = 15 \end{cases}$$

- Este puede escribirse como:

$$\begin{aligned} x_1 &= \frac{7 + x_2 - x_3}{4} \\ x_2 &= \frac{21 + 4x_1 + x_3}{8} \\ x_3 &= \frac{15 + 2x_1 - x_2}{5} \end{aligned}$$

Metódos iterativos: *Jacobi*

- Lo cual sugiere el esquema iterativo:

$$\begin{aligned}
 x_1^{(k)} &= \frac{7 + x_2^{(k-1)} - x_3^{(k-1)}}{4} \\
 x_2^{(k)} &= \frac{21 + 4x_1^{(k-1)} + x_3^{(k-1)}}{8} \\
 x_3^{(k)} &= \frac{15 + 2x_1^{(k-1)} - x_2^{(k-1)}}{5}
 \end{aligned}
 \quad k \geq 1$$

- Necesitamos un vector de aproximación inicial:

$$\mathbf{x}_0 = \left[x_1^{(0)}, x_2^{(0)}, x_3^{(0)} \right]^T$$

- Este esquema se conoce como **método iterativo de Jacobi**

Metódos iterativos: *Jacobi*

- Tomando como vector inicial $\mathbf{x}_0 = [1, 2, 2]^T$:

$$x_1^{(1)} = \frac{7 + 2 - 2}{4} = 1,75$$

$$x_2^{(1)} = \frac{21 + 4 \cdot (1) + 2}{8} = 3,375$$

$$x_3^{(1)} = \frac{15 + 2 \cdot (1) - 2}{5} = 3$$

- Luego

$$x_1^{(2)} = \frac{7 + 3,375 - 3}{4} = 1,84375$$

$$x_2^{(2)} = \frac{21 + 4 \cdot (1,75) + 3}{8} = 3,875$$

$$x_3^{(2)} = \frac{15 + 2 \cdot (1,75) - 3,375}{5} = 3,205$$

Metódos iterativos: *Jacobi*

- Obtenemos así la sucesión de vectores:

$$\mathbf{x}_0 = \begin{bmatrix} x_1^{(0)} & x_2^{(0)} & x_3^{(0)} \end{bmatrix}^T = [1,0000, 2,0000, 2,0000]^T$$

$$\mathbf{x}_1 = \begin{bmatrix} x_1^{(1)} & x_2^{(1)} & x_3^{(1)} \end{bmatrix}^T = [1.7500, 3.3750, 3.0000]^T$$

$$\mathbf{x}_2 = \begin{bmatrix} x_1^{(2)} & x_2^{(2)} & x_3^{(2)} \end{bmatrix}^T = [1.8438, 3.8750, 3.2050]^T$$

$$\mathbf{x}_3 = \begin{bmatrix} x_1^{(3)} & x_2^{(3)} & x_3^{(3)} \end{bmatrix}^T = [1.9625, 3.9250, 2.9625]^T$$

$$\mathbf{x}_4 = \begin{bmatrix} x_1^{(4)} & x_2^{(4)} & x_3^{(4)} \end{bmatrix}^T = [1.9906, 3.9766, 3.0000]^T$$

$$\mathbf{x}_5 = \begin{bmatrix} x_1^{(5)} & x_2^{(5)} & x_3^{(5)} \end{bmatrix}^T = [1.9941, 3.9953, 3.0009]^T$$

$$\vdots \qquad \qquad \qquad \vdots \qquad \qquad \qquad \vdots$$

$$\mathbf{x}_{10} = \begin{bmatrix} x_1^{(10)} & x_2^{(10)} & x_3^{(10)} \end{bmatrix}^T = [2.0000, 4.0000, 3.0000]^T$$

Metódos iterativos: *Jacobi*

- Dado el SEL $\mathbf{Ax} = \mathbf{b}$

$$\begin{cases} x_1^{(k)} = \left(b_1 - \sum_{j=1, j \neq 1}^n a_{1j} x_j^{(k-1)} \right) / a_{11} \\ x_2^{(k)} = \left(b_2 - \sum_{j=1, j \neq 2}^n a_{2j} x_j^{(k-1)} \right) / a_{22} \\ \vdots \\ x_n^{(k)} = \left(b_n - \sum_{j=1, j \neq n}^n a_{nj} x_j^{(k-1)} \right) / a_{nn} \end{cases}$$

- En general para $i = 1, \dots, n$

$$x_i^{(k)} = \left(b_i - \sum_{j=1, j \neq i}^n a_{ij} x_j^{(k-1)} \right) / a_{ii} \quad i =, \dots, n$$

para $k \geq 1$

La implementación *Jacobi.m/.sci*

```
function [x,iter] = Jacobi(A,b,x0,tol,itmax)
% Método iterativo de Jacobi
% A : matriz invertible nxn
% b : lado derecho n x 1
% x0 : estimación inicial
% tol : tolerancia
% itmax : número máximo de iteraciones
% x : vector nx1 aproximacion solución SEL Ax=b
n = length(b);
x0=x0';
if diagdom(A) == 0
    disp('No se garantiza convergencia');
end
...
```

La implementación *Jacobi.m/.sci*

```
...  
iter = 0;  
while 1  
    iter=iter + 1;  
    for i =1:n  
        x(i) = b(i);  
        for j=1:n  
            if j == i  
                x(i) = x(i) - A(i,j)*x0(j);  
            end  
        end  
        x(i) = x(i) / A(i,i);  
    end  
...  

```

¿Cuándo cortamos *Jacobi*?

- Tenemos dos criterios de corte o de parada:

```
...
err = normap(x-x0,2);
if err < tol
    x=x', break,
end
if iter == itmax
    disp('Número máximo de iteraciones alcanzado');
    x=x', break
end
x0 = x;
end
end
```

Ejemplo de Jacobi

```
>> A = [ 4 -1 1; 4 -8 1; -2 1 5 ];  
>> b = [ 7 -21 15 ]';  
>> x0 = [ 1 2 2 ]';  
>> tol = 1e-4;  
>> itmax = 30;  
>> [xJ,iterJ] = Jacobi(A,b,x0,tol,itmax)
```

```
xJ =  
2.0000  
4.0000  
3.0000
```

```
iterJ =  
10
```

Alcanza la tolerancia en 10 iteraciones.

MÉTODO DE GAUSS-SEIDEL

Metódos iterativos: *Gauss-Seidel*

- Para mejorar la convergencia del método de Jacobi modificamos el esquema iterativo:

$$\begin{aligned}
 x_1^{(k)} &= \frac{7 + x_2^{(k-1)} - x_3^{(k-1)}}{4}, \\
 x_2^{(k)} &= \frac{21 + 4x_1^{(k)} + x_3^{(k-1)}}{8}, \\
 x_3^{(k)} &= \frac{15 + 2x_1^{(k)} - x_2^{(k)}}{5}.
 \end{aligned}
 \quad k \geq 1,$$

Los supraíndices en **verde** son la diferencia con Jacobi.

- Este esquema se conoce como **método iterativo de Gauss-Seidel**.

Metódos iterativos: *Gauss-Seidel*

- Dado el SEL $\mathbf{Ax} = \mathbf{b}$

$$\begin{cases} x_1^{(k)} &= \left(b_1 - \sum_{j=2}^n a_{1j} x_j^{(k-1)} \right) / a_{11}, \\ x_2^{(k)} &= \left(b_2 - a_{21} x_1^{(k)} - \sum_{j=3}^n a_{2j} x_j^{(k-1)} \right) / a_{22}, \\ x_3^{(k)} &= \left(b_3 - a_{31} x_1^{(k)} - a_{32} x_2^{(k)} - \sum_{j=4}^n a_{3j} x_j^{(k-1)} \right) / a_{33}, \\ &\vdots \\ x_n^{(k)} &= \left(b_n - \sum_{j=1}^{n-1} a_{nj} x_j^{(k)} \right) / a_{nn}. \end{cases}$$

- En general, para $i = 1, \dots, n$

$$x_i^{(k)} = \frac{1}{a_{ii}} \left(b_i - a_{i1} x_1^{(k)} - \dots - a_{ii-1} x_{i-1}^{(k)} - a_{ii+1} x_{i+1}^{(k-1)} - \dots - a_{in} x_n^{(k-1)} \right),$$

con $k \geq 1$.

Metódos iterativos: *Gauss-Seidel*

- Usando la misma aproximación inicial $\mathbf{x}_0 = [1, 2, 2]^T$ obtenemos la sucesión:

$$\mathbf{x}_0 = [x_1^{(0)}, x_2^{(0)}, x_3^{(0)}]^T = [1.0000, 2.0000, 2.0000]^T$$

$$\mathbf{x}_1 = [x_1^{(1)}, x_2^{(1)}, x_3^{(1)}]^T = [1.7500, 3.3750, 2.9200]^T$$

$$\mathbf{x}_2 = [x_1^{(2)}, x_2^{(2)}, x_3^{(2)}]^T = [1.9500, 3.9688, 2.9862]^T$$

$$\mathbf{x}_3 = [x_1^{(3)}, x_2^{(3)}, x_3^{(3)}]^T = [1.9956, 3.9961, 2.9990]^T$$

$$\mathbf{x}_4 = [x_1^{(4)}, x_2^{(4)}, x_3^{(4)}]^T = [1.9993, 3.9995, 2.9998]^T$$

$$\mathbf{x}_5 = [x_1^{(5)}, x_2^{(5)}, x_3^{(5)}]^T = [1.9999, 3.9999, 3.0000]^T$$

$$\mathbf{x}_6 = [x_1^{(6)}, x_2^{(6)}, x_3^{(6)}]^T = [2.0000, 4.0000, 3.0000]^T$$

La implementación *GaussSeidel.m/.sci*

```
function [x,iter] = GaussSeidel(A,b,x0,tol,itmax)
% Metodo iterativo de Gauss-Seidel
% A : matriz invertible nxn
% b : lado derecho n x 1
% x0 : estimacion inicial
% tol : tolerancia
% itmax : numero maximo de iteraciones
% x : vector nx1 aproximacion solucion SEL Ax=b
... (idem Jacobi)
for i=1:n
x(i) = b(i);
for j=1:n
if j == i
x(i) = x(i) - A(i,j)*x0(j); % diferencia con Jacobi
end
end
x(i) = x(i) / A(i,i);
end
... (idem Jacobi)
```

Ejemplo de Gauss-Seidel

```
>> A = [ 4 -1 1; 4 -8 1; -2 1 5 ];  
>> b = [ 7 -21 15 ]';  
>> x0 = [ 1 2 2 ]';  
>> tol = 1e-4;  
>> itmax = 30;  
>> [xGS,iterGS] = GaussSeidel(A,b,x0,tol,itmax)
```

```
xGS =  
2.0000  
4.0000  
3.0000
```

```
iterGS =  
6
```

Alcanza la tolerancia en 6 iteraciones.

MÉTODO DE SOR

La iteración de SOR (Successive overrelaxation)

- El método de la sobrerelajación sucesiva (SOR) fue desarrollado para acelerar la convergencia del método de Gauss-Seidel.
- La idea es formar sucesivamente un peso ponderado entre valores calculados anteriormente.
- En G-S, para $i = 1, \dots, n$

$$x_i^{(k)} = \frac{1}{a_{ii}} \left(b_i - a_{i1}x_1^{(k)} - \dots - a_{ii-1}x_{i-1}^{(k)} - a_{ii+1}x_{i+1}^{(k-1)} - \dots - a_{in}x_n^{(k-1)} \right),$$

con $k \geq 1$ (paso).

- En términos de sumatoria, tenemos:

$$x_i^{(k)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j=1}^{i-1} a_{ij}x_j^{(k)} - \sum_{j=i+1}^n a_{ij}x_j^{(k-1)} \right),$$

con $k \geq 1$.

La iteración de SOR (Successive overrelaxation)

- En SOR se pondera el “nuevo valor calculado” un factor ω y el “valor anterior” se pondera con un factor $(1 - \omega)$.
- Asumiendo que las sumatorias son ignoradas cuando $i = 1$ y n , respectivamente, tenemos:

$$x_i^{(k)} = \frac{\omega}{a_{ii}} \left(b_i - \sum_{j=1}^{i-1} a_{ij} x_j^{(k)} - \sum_{j=i+1}^n a_{ij} x_j^{(k-1)} \right) + (1 - \omega) x_i^{(k-1)},$$

con $k \geq 1$.

- Para que este método proporcione una mejora sobre Gauss-Seidel, ω debe elegirse cuidadosamente.
- ω se denomina *parámetro de relajación*.
- Si $\omega = 1$, SOR y Gauss-Seidel son idénticos.
- Si $\omega > 1$, se dice que la relajación es excesiva (overrelaxation).
- Si $\omega < 1$, se dice que la relajación es insuficiente (underrelaxation.).

La implementación *SOR.m/.sci*

```
function [x,iter,resrel] = SOR(A,b,x0,omega,tol,itmax)
% Metodo iterativo de Sobrerelajacion sucesiva (SOR)
% Input :
% A : matriz invertible nxn
% b : lado derecho n x 1
% x0 : estimacion inicial
% omega : parametro de relajacion
% tol : tolerancia
% itmax : numero maximo de iteraciones
% Output:
% x : vector nx1 aproximacion solucion SEL Ax=b
% iter : numero de iteraciones realizadas
% resrel: residuo relativo a norm(b)
[m,n] = size(A);
if m~=n
    disp('La matriz A no es cuadrada.')
return;
end
```

La implementación *SOR.m/.sci*

```

...
k = 1;
x = x0;
for k = 1:itmax
    x(1) = omega*(1/A(1,1))*(b(1)-A(1,2:n)*x(2:n)) + (1-omega)*x(1);
    for i = 2:n-1
        x(i) = omega*(1/A(i,i))*(b(i) - ...
            A(i,1:i-1)*x(1:i-1)-A(i,i+1:n)*x(i+1:n)) + (1-omega)*x(i);
    end
    x(n) = omega*(1/A(n,n))*(b(n) - A(n,1:n-1)*x(1:n-1)) + (1-omega)*x(n);
    resrel = norm(b-A*x)/norm(b); % residuo residual relativo por iteracion
    if resrel < tol
        iter = k;
        return;
    end
end
iter = -1; % Si el metodo de SOR no converge, iter = -1.
end

```

No es el mismo criterio de parada que Jacobi/G-S.

Metódos iterativos: *SOR*

- Usando la misma aproximación inicial $\mathbf{x}_0 = [1, 2, 2]^T$ y $\omega = 1$ obtenemos la sucesión:

$$\mathbf{x}_0 = \begin{bmatrix} x_1^{(0)} & x_2^{(0)} & x_3^{(0)} \end{bmatrix}^T = [1.0000, 2.0000, 2.0000]^T$$

$$\mathbf{x}_1 = \begin{bmatrix} x_1^{(1)} & x_2^{(1)} & x_3^{(1)} \end{bmatrix}^T = [1.7500, 3.3750, 2.9200]^T$$

$$\mathbf{x}_2 = \begin{bmatrix} x_1^{(2)} & x_2^{(2)} & x_3^{(2)} \end{bmatrix}^T = [1.9500, 3.9688, 2.9862]^T$$

$$\mathbf{x}_3 = \begin{bmatrix} x_1^{(3)} & x_2^{(3)} & x_3^{(3)} \end{bmatrix}^T = [1.9956, 3.9961, 2.9990]^T$$

$$\mathbf{x}_4 = \begin{bmatrix} x_1^{(4)} & x_2^{(4)} & x_3^{(4)} \end{bmatrix}^T = [1.9993, 3.9995, 2.9998]^T$$

$$\mathbf{x}_5 = \begin{bmatrix} x_1^{(5)} & x_2^{(5)} & x_3^{(5)} \end{bmatrix}^T = [1.9999, 3.9999, 3.0000]^T$$

Metódos iterativos: *SOR*

- Usando la misma aproximación inicial $\mathbf{x}_0 = [1, 2, 2]^T$ y $\omega = 1.5$ obtenemos la sucesión:

$$\mathbf{x}_0 = [x_1^{(0)}, x_2^{(0)}, x_3^{(0)}]^T = [1.0000, 2.0000, 2.0000]^T$$

$$\mathbf{x}_1 = [x_1^{(1)}, x_2^{(1)}, x_3^{(1)}]^T = [1.7500, 3.3750, 2.9200]^T$$

$$\mathbf{x}_2 = [x_1^{(2)}, x_2^{(2)}, x_3^{(2)}]^T = [1.9500, 3.9688, 2.9862]^T$$

$$\mathbf{x}_3 = [x_1^{(3)}, x_2^{(3)}, x_3^{(3)}]^T = [1.9956, 3.9961, 2.9990]^T$$

$$\mathbf{x}_4 = [x_1^{(4)}, x_2^{(4)}, x_3^{(4)}]^T = [1.9993, 3.9995, 2.9998]^T$$

$$\vdots \qquad \qquad \qquad \vdots \qquad \qquad \qquad \vdots$$

$$\mathbf{x}_{13} = [x_1^{(13)}, x_2^{(13)}, x_3^{(13)}]^T = [2.0002, 4.0001, 3.0001]^T$$

Metódos iterativos: *SOR*

- Usando la misma aproximación inicial $\mathbf{x}_0 = [1, 2, 2]^T$ y $\omega = 0.5$ obtenemos la sucesión:

$$\mathbf{x}_0 = [x_1^{(0)}, x_2^{(0)}, x_3^{(0)}]^T = [1.0000, 2.0000, 2.0000]^T$$

$$\mathbf{x}_1 = [x_1^{(1)}, x_2^{(1)}, x_3^{(1)}]^T = [1.7500, 3.3750, 2.9200]^T$$

$$\mathbf{x}_2 = [x_1^{(2)}, x_2^{(2)}, x_3^{(2)}]^T = [1.9500, 3.9688, 2.9862]^T$$

$$\mathbf{x}_3 = [x_1^{(3)}, x_2^{(3)}, x_3^{(3)}]^T = [1.9956, 3.9961, 2.9990]^T$$

$$\mathbf{x}_4 = [x_1^{(4)}, x_2^{(4)}, x_3^{(4)}]^T = [1.9993, 3.9995, 2.9998]^T$$

$$\vdots \qquad \qquad \qquad \vdots \qquad \qquad \qquad \vdots$$

$$\mathbf{x}_{18} = [x_1^{(18)}, x_2^{(18)}, x_3^{(18)}]^T = [1.9997, 3.9996, 2.9999]^T$$

Metódos iterativos: *SOR*

- Usando la misma aproximación inicial $\mathbf{x}_0 = [1, 2, 2]^T$ y $\omega = 1.05$ obtenemos la sucesión:

$$\mathbf{x}_0 = \begin{bmatrix} x_1^{(0)} & x_2^{(0)} & x_3^{(0)} \end{bmatrix}^T = [1.0000, 2.0000, 2.0000]^T$$

$$\mathbf{x}_1 = \begin{bmatrix} x_1^{(1)} & x_2^{(1)} & x_3^{(1)} \end{bmatrix}^T = [1.7500, 3.3750, 2.9200]^T$$

$$\mathbf{x}_2 = \begin{bmatrix} x_1^{(2)} & x_2^{(2)} & x_3^{(2)} \end{bmatrix}^T = [1.9500, 3.9688, 2.9862]^T$$

$$\mathbf{x}_3 = \begin{bmatrix} x_1^{(3)} & x_2^{(3)} & x_3^{(3)} \end{bmatrix}^T = [1.9956, 3.9961, 2.9990]^T$$

$$\mathbf{x}_4 = \begin{bmatrix} x_1^{(4)} & x_2^{(4)} & x_3^{(4)} \end{bmatrix}^T = [1.9997, 4.0000, 2.9998]^T$$

¿Cómo elegimos el parámetro ω ?

Ejemplo de SOR

```
>> A = [ 4 -1 1; 4 -8 1; -2 1 5 ];
>> b = [ 7 -21 15 ]';
>> x0 = [ 1 2 2 ]';
>> tol = 1e-4;
>> itmax = 30;
>> omega = 1.05;
>> [xSOR,iterSOR,resrelSOR] = SOR(A,b,x0,omega,tol,itmax)
```

xSOR =

1.9997

4.0000

2.9998

iterSOR =

4 **Alcanza la tolerancia en 4 iteraciones.**

resrelSOR =

7.2778e-05

CONVERGENCIA DE JACOBI Y GAUSS-SEIDEL

¿Cuándo convergen estos esquemas iterativos?

Definición. Una matriz $\mathbf{A}$ $n \times n$ es **diagonalmente dominante** sii:

$$|a_{ii}| > \sum_{j=1, j \neq i}^n |a_{ij}| \quad i = 1, \dots, n$$

EJEMPLO: la matriz anterior es diagonalmente dominante pues

$$|4| > |-1| + |1|$$

$$|-8| > |4| + |1|$$

$$|5| > |-2| + |1|$$

Teorema. Si $\mathbf{A}$ es una matriz diagonalmente dominante, los métodos iterativos de Jacobi y Gauss-Seidel convergen a la solución del SEL $\mathbf{Ax} = \mathbf{b}$ para cualquier vector inicial $\mathbf{x}_0$

Demo. No la hacemos. Consultar [MaFi2000].

¿Cuándo convergen estos esquemas iterativos?

```
function z = diagdom(A)
% Determina si la matriz A es diagonalmente dominante
% Datos: A matriz invertible nxn
% Resultado: z variable logica: 0 false - 1 true
n = length(A);
i = 0;
z = 1;
while ( i < n && z == 1 )
    i = i+1;
    if sum(abs(A(i,:))) >= 2*abs(A(i,i))
        z = 0;
    end
end
end
```


Observaciones sobre la convergencia

Si **A** matriz inversible con diagonal no nula del SEL **Ax = b**:

- No siempre está garantizada la convergencia de las esquemas iterativos de Jacobi y Gauss-Seidel
- Las iteraciones de Jacobi y G-S aplicadas al mismo SEL pueden ser:
 - **ambas convergentes,**
 - **ambas divergentes,**
 - **o bien una cualquiera de ellas convergente y la otra no**
- Cuando las iteraciones de Jacobi y G-S resultan ambas convergentes, **en general, pero no siempre, las iteraciones de Gauss-Seidel convergen más rápidamente que las de Jacobi**

Comparar qué sucede en el ejemplo anterior

El vector residual y el vector error

- Al resolver en una máquina $\mathbf{Ax} = \mathbf{b}$ con precisión finita, obtenemos una aproximación $\tilde{\mathbf{x}}$ del vector solución $\mathbf{x}$:

$$\tilde{\mathbf{x}} = \mathbf{x} + \Delta \mathbf{x}$$

¿Cuánto difiere nuestra solución $\tilde{\mathbf{x}}$ de la solución exacta $\mathbf{x}$?

Definición. Llamaremos **vector residual** al vector:

$$\mathbf{r} = \mathbf{b} - \mathbf{A}\tilde{\mathbf{x}}$$

y **vector error** al vector:

$$\mathbf{e} = \mathbf{x} - \tilde{\mathbf{x}}$$

Nota. Se puede verificar la convergencia de estos métodos calculando el vector residual o su norma. En general, una norma residual casi nula equivale a una aproximación buena aunque esto no es una regla general.

CONVERGENCIA DE SOR

La elección de ω para el SOR

- Una mala elección del parámetro de relajación ω puede conducir a tasas de convergencia pobres.
- Una buena elección de ω puede conducir a una convergencia rápida en comparación con Jacobi o Gauss-Seidel.
- Encontrar el valor óptimo de ω_{opt} es difícil en general y sólo se conoce para tipos especiales de matrices.
- Sin embargo se sabe que ω debe satisfacer $0 < \omega < 2$.

Teorema. Si la iteración de SOR converge a la solución del SEL $\mathbf{Ax} = \mathbf{b}$ para cualquier vector inicial $\mathbf{x}_0$, entonces $0 < \omega < 2$.

Demo. No la hacemos. Consultar [Ford2014].

- Este resultado dice que nunca se elige ω fuera del intervalo $(0, 2)$.
- Normalmente la elección es $\omega > 1$ (sobre-relajación) para dar el mayor peso a los nuevos valores calculados.

CONDICIONAMIENTO

Sistemas mal condicionados

Definición. Un SEL $\mathbf{Ax} = \mathbf{b}$ se dice que está **mal condicionado**, si pequeños cambios en la matriz de coeficientes $\mathbf{A}$ o del vector $\mathbf{b}$, generan grandes cambios en el vector solución $\mathbf{x}$.

- ① **Caso 1.** Modificamos solo el vector del lado derecho:

$$\mathbf{A}(\mathbf{x} + \Delta\mathbf{x}) = (\mathbf{b} + \Delta\mathbf{b})$$

- ② **Caso 2.** Modificamos solo la matriz de coeficientes:

$$(\mathbf{A} + \Delta\mathbf{A})(\mathbf{x} + \Delta\mathbf{x}) = \mathbf{b}$$

- ③ **Caso 3.** Modificamos ambos:

$$(\mathbf{A} + \Delta\mathbf{A})(\mathbf{x} + \Delta\mathbf{x}) = (\mathbf{b} + \Delta\mathbf{b})$$

¿Cómo medimos el condicionamiento del SEL?

Número de condición de una matriz

Definición. Siendo **A** una matriz inversible, llamamos **número de condición** al número:

$$\kappa(\mathbf{A}) = \|\mathbf{A}\| \cdot \|\mathbf{A}^{-1}\|$$

donde la norma $\|\cdot\|$ satisface la *condición de compatibilidad*

EJEMPLO: en MATLAB/Octave/Scilab usamos **cond** para calcularlo en norma 2 o **rcond** para calcular su recíproco $1/\kappa(\mathbf{A})$ en norma 1

```
>> A = [4 -1 1; 4 -8 1; -2 1 5];
>> k = cond(A)
k = 3.1467
>> B = [4.1 2.8; 9.7 6.6]
>> k = cond(B)
k = 1.6230e+03
```

La matriz de Hilbert

EJEMPLO: Las matrices de Hilbert H_n de $n \times n$ están definidas como

$$H_n(i,j) = \frac{1}{i+j-1}, \quad 1 \leq i,j \leq n.$$

Por ejemplo, si $n = 5$

$$H_5 = \begin{bmatrix} 1 & 1/2 & 1/3 & 1/4 & 1/5 \\ 1/2 & 1/3 & 1/4 & 1/5 & 1/6 \\ 1/3 & 1/4 & 1/5 & 1/6 & 1/7 \\ 1/4 & 1/5 & 1/6 & 1/7 & 1/8 \\ 1/5 & 1/6 & 1/7 & 1/8 & 1/9 \end{bmatrix}.$$

MATLAB/Octave usamos **hilb** y en Scilab **inv(testmatrix('hilb',n))**:

```
>> H5 = hilb(5);
>> cond(H5)
ans = 4.7661e+05
>> cond(hilb(10))
ans = 1.6025e+13
```

¡El número de condición $\kappa(H_n)$ crece rápido!

Observaciones sobre $\kappa(\mathbf{A})$

Teorema. El error relativo al resolver $\mathbf{Ax} = \mathbf{b}$ está acotado por:

$$\frac{\|\mathbf{x} - \tilde{\mathbf{x}}\|}{\|\mathbf{x}\|} \leq \kappa(\mathbf{A}) \frac{\|\mathbf{b} - \mathbf{A}\tilde{\mathbf{x}}\|}{\|\mathbf{b}\|}$$

Demo. No la hacemos. Consultar [Mole2008].

- $\kappa(\mathbf{A}) \geq 1$ para toda matriz $\mathbf{A}$ inversible y norma compatible.
- Si $\kappa(\mathbf{A})$ es "pequeño" $\implies$ el error relativo también lo será.
Diremos que $\mathbf{A}$ **está bien condicionada**.
- Si $\kappa(\mathbf{A})$ es "grande" $\implies$ el error relativo también lo será.
Diremos que $\mathbf{A}$ **está mal condicionada**.

*Cuando la matriz está muy mal condicionada ($\text{cond}(\mathbf{A}) \approx 1/\text{eps}$),
puede haber pérdida de precisión en el cálculo de la solución $\mathbf{x}$.*

Bibliografía

- **[Ford2015]** W. Ford, *Numerical Linear Algebra with Applications using MATLAB*, 1st Ed., Elsevier, USA, 2015.
- **[MaFi2000]** J.M. Mathews - K.D. Fink, *Métodos Numéricos con MATLAB*, 3era Ed., Pearson Prentice Hall, Madrid, 2000.
- **[Ponz2024]** L. Ponzellini Marinelli, *Taller de Álgebra Lineal en Scilab*, FCEIA, UNR, 2024.
- **[SPMK2023]** J.A. Semitiel, L. Ponzellini Marinelli & A.I. Kurdobrin, *Notas de Clases de Matemática Aplicada de Ingeniería Eléctrica e Ingeniería Electrónica*, FCEIA, UNR, 2023.
- **[SiSo2010]** J.W. Signorelli, J. Sorribas, *Notas de Clases de Informática Aplicada y Métodos Computacionales*, FCEIA, UNR, 2010.
- **[TrBa1997]** L.I.N. Trefethen - D. Bau, *Numerical Linear Algebra*, SIAM, Philadelphia, 1997.